

Skip List
–Display List
–Space Complexity Analysis

Program:

```
// Display the skip list layer by layer
void displayList(struct SkipList *lst) {
    cout << "\n--- Skip List ---" << endl;
    for (int i = 0; i <= lst->level; i++) {
        struct Node *node = lst->header->forward[i];
        cout << "Level " << i << ": ";
        while (node != NULL) {
            cout << node->key << " ";
            node = node->forward[i];
        }
        cout << endl;
    }
    cout << "-----" << endl;
}

case 4:
    displayList(list);
    break;
```

Space Complexity Analysis

1. Input Space Complexity

The input space is evaluated by looking at the specific arguments passed into the function and the total size of the data structure it operates on.

A. Parametric Input Space: $O(1)$

- ***The Parameter:*** *The function signature is `void displayList(struct SkipList * lst)`.*

It only takes a single argument: a memory pointer addressing the existing Skip List structure.

- ***The Math:*** *A standard memory pointer takes a fixed, tiny amount of memory (typically 8 bytes on a 64 – bit system).*
- ***The Conclusion:*** *Because the size of this pointer never changes regardless of how many millions of elements are inside the list, the parametric input space is strictly $O(1)$.*

B. Full Data Holistic Input Space: $O(n)$

- ***The Structure:*** *The function is tasked with displaying the entire, fully – built Skip List.*

- **The Math:** As proven in the Space Complexity document, a Skip List using a 50% coin flip probability ($p = 0.5$) generates an average of 2 pointers per node. Therefore, a list of n elements will contain an expected $2n$ total pointers.

Method 1: The Vertical Approach (Expected Pointers per Node)

Instead of looking at the whole list, this method asks:

When a single node is created, how many pointers

does it have on average?

Because the node's height is decided by a 50% coin flip ($p = 0.5$), we calculate the probability of it getting a pointer on each specific level:

- Level 0: It ALWAYS gets a pointer here. (**Probability = 1**).
- Level 1: It gets a pointer here if the first coin flip is Heads. (**Probability = p**).
- Level 2: It gets a pointer here if it flips Heads twice in a row. (**Probability = p^2**).
- Level 3: It gets a pointer here if it flips Heads three times. (**Probability = p^3**).

To find the total expected pointers for one single node,

we sum this infinite geometric series:

$$\mathbb{E}[\text{Pointers per Node}] = 1 + p + p^2 + p^3 + \dots = S_{\infty} = \frac{1}{1 - p}$$

Now, substitute $p = 0.5$ (for our 50% coin flip):

$$\frac{1}{1 - 0.5} = \frac{1}{0.5} = 2$$

- **The Result:** A single node has an average of exactly **2** pointers.
- **Total Calculation:** If **1** node has **2** pointers, then n elements require exactly **$2n$** total pointers.

Method 2: The Horizontal Approach (Elements per Layer)

This method looks at the entire Skip List horizontally, summing up the total number of nodes expected to exist on every single floor.

- **Level 0:** Must contain every single element. (Total pointers = n).
- **Level 1:** Because of the 50% coin flip, only half of Level 0 is expected to be promoted. (Total expected pointers = $\frac{n}{2}$).
- **Level 2:** Only half of Level 1 is expected to be promoted. (Total expected pointers = $\frac{n}{4}$).
- **Level 3:** Half of Level 2. (Total expected pointers = $\frac{n}{8}$).

To find the total memory space, we add all the floors together:

$$\text{Total Space} = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots$$

If we factor out the n , it looks like this:

$$\text{Total Space} = n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right)$$

The sum of that infinite fraction series inside the parentheses

*$\left(1 + \frac{1}{2} + \frac{1}{4} \dots \right)$ mathematically resolves to exactly **2**.*

$$\text{Total Space} = n \times 2 = 2n$$

Both methods perfectly mathematically align to prove that a

*Skip List with n elements will generate exactly **$2n$** pointers on average!*

2. Auxiliary Space Complexity: $O(1)$

Auxiliary space measures the temporary, extra memory the algorithm requests specifically to do its job, which is then destroyed when the function finishes.

- **The Variables:** Inside the `displayList` function, the code only declares two temporary variables: an integer `i` for the `for` loop, and a traversal pointer `struct Node * node` to walk across the levels.
- **No Arrays Needed:** Unlike the Insertion and Deletion modules, the Display module does not need to remember its previous path. Therefore, it does not allocate the `update[MAX_LEVEL + 1]` array.
- **The Conclusion:** Because the function allocates zero arrays, makes zero recursive calls, and only uses two simple temporary variables, its auxiliary space overhead is an absolute, strict $O(1)$.

Final Total Space Complexity

To find the total space complexity of the module, we add the dominant input space to the auxiliary space:

- **Total Space Complexity** = Full Data Holistic Space: $(O(n))$ + Auxiliary Space: $(O(1)) = O(n)$.
-

Algorithm: SkipListDisplay(LST)

- [illegible]